Polymarket Unified SDK: Design Reference

> *This is the document I used for collecting various references to make sure AI agents working on this codebase understand the context and have enough background. It's the long-form version of design.md, with full attribution.*

A personal reference for the design choices on this prototype. Captures the WHY behind each decision, what I learned from other SDKs, and where each pattern came from.

Status: Design locked. Ready to implement.
Last updated: 2026-04-29

1. The problem I'm solving

Polymarket's developer surface is fragmented across three hosts, three concept models, and two auth layers:

Host	Owns	Auth	SDK
`gamma-api.polymarket.com`	Markets, events, slugs, outcomes, `clobTokenIds` mapping	None	None (REST)
`clob.polymarket.com`	Orderbook, midpoint, spread, `calculateMarketPrice`, order placement, EIP-712 signing	L1 (EIP-712) → derive creds → L2 (HMAC)	`@polymarket/clob-client-v2`
`data-api.polymarket.com`	Positions, activity, value, leaderboards	None	None (REST)

Plus auxiliary services (Bridge for deposits, Relayer for gasless txs) and one in-band concept (Safe proxy / `signatureType: 2`) that the journey touches.

To complete the four-step journey (fetch market → quote → place → view positions), a developer currently needs to learn:

- Which service owns which concept
- That `conditionId` ≠ `clobTokenId` (markets vs tokens)
- Outcome → token disambiguation ("Yes"/"No" or candidate name; `clobTokenIds[i]` aligns with `outcomes[i]`)
- That `signatureType: 2` means Safe proxy and `funder` is the proxy address, not the EOA `signer`
- That FOK `amount` flips by side: USDC for BUY, shares for SELL
- That `price` on a market order is a worst-price *cap*, not a target
- The `negRisk` flag per market (different exchange contracts)
- Tick size per market
- That API creds are derived once via L1 then cached and re-used for L2

None of these concepts are wrong to surface. Most of them are plumbing for the journey, though, not concepts the developer wants to think about per call. The SDK's job is to hide them from the journey path while still naming them in types and docs.

2. The journey this SDK supports

1. **Fetch a market** by slug (Gamma API)
2. **Estimate** the FOK market-order quote (CLOB)
3. **Place** the FOK order (CLOB)
4. **View** resulting positions (Data API)

Scenario assumptions I've locked in: Polygon mainnet, `signatureType: 2` (Safe proxy), order type `FOK`, signer = EOA private key, funder = Polymarket Safe proxy address. Configurable via override, but not the default path.

3. Design pillars

1. **Workflow-first surface, service-second:** Resource methods on the facade are the headline. Sub-clients exist as escape hatches.
2. **Don't pretend concepts don't exist:** The README and types still acknowledge funder/signer, conditionId/tokenId, sigType 2, neg-risk. I just hide them from the journey path.
3. **Default-safe:** Live trading requires double-gating. Dry-run is a first-class mode.
4. **Testable by construction:** Every external integration sits behind a structurally-typed seam.
5. **Tolerant on input, strict on output:** Gamma payloads are messy; I normalize at the edge into stable types. Errors are named subclasses, never strings.

4. What I learned from other SDKs

Stripe: *the gold standard for DX*

Lesson	Where it applies
Consistency across the abstraction ladder: REST → SDK → framework SDK should expose the same names and shapes.	Resource-oriented namespace (`client.markets.get` , `client.orders.place`) maps 1:1 to Polymarket's REST organization (`/markets/` , `/orders/` , `/positions/`).
Inspectability: Stripe's request logs let developers verify what hit the API.	`dryRun: true` returns the signed EIP-712 payload for inspection before live submission.
Progressive complexity: Defaults handle 80% of cases; opting into power is explicit.	Slippage defaults to 100 bps; tick rounding direction is automatic; sigType 2 is the default. You opt out, not in.
Code-generated types from a single OpenAPI source.	Aspirational since I don't codegen here; I hand-normalize. Worth noting as a future direction.

Sources: Stripe SDKs, Auchenberg's DX writeup, Moesif teardown.

CCXT: *unification across heterogeneous backends*

Lesson	Where it applies
Normalize the data shape across backends with different response formats.	One `Market` type across Gamma + CLOB; one `Position` type from Data API; tolerant field parsing handles three Gamma response shapes (single market, market list, event-with-nested-markets).
OO base class + adapters with parallel method names.	Sub-clients (`GammaClient` , `ClobAdapter` , `DataApiClient`) share a uniform shape (same constructor signature, same error semantics) even though the underlying services don't.

Source: CCXT docs, GitHub.

viem: *naming and account abstraction*

Lesson	Where it applies
Avoid the word "Signer" because locally-held keys and JSON-RPC accounts have different security and ergonomics. viem prefers Wallet/Account terminology.	I take a viem `Account` (LocalAccount or JsonRpcAccount) as input. I never invent a custom `Signer` abstraction.
Client objects per role (Public/Wallet/Test).	Maps to my split: read-only `GammaClient` / `DataApiClient` vs write-capable `ClobAdapter`.

Source: viem, account abstraction.

Hyperliquid (`@nktkas/hyperliquid` , QuickNode SDK): *one-liner trading ergonomics*

```
sdk.marketBuy("BTC", { size: 0.001 })
sdk.marketSell("ETH", { notional: 100 })
```

Lesson	Where it applies
Symbol-keyed market orders so the caller never types raw asset IDs.	I use outcome name (`"Yes"` , `"No"` , candidate name) instead of raw `clobTokenId` . Raw IDs stay accessible for advanced use.
Notional vs size split mirrors FOK semantics	Polymarket's FOK `amount` is USDC for BUY, shares for SELL. Same idea, surfaced as `amountUsd` / `shares` in my types.

Source: @nktkas/hyperliquid.

`HuakunShen/polymarket-kit` : *prior art (the contrast that matters)*

This library already exists as "a fully typed SDK with unified clients for CLOB, Gamma API, and Data API." It exposes three separate sub-clients: `GammaSDK` , `PolymarketSDK` , `DataSDK` .

But it explicitly **lacks orchestration**. The architecture review notes:

> "A user must manage three client instances and pass data between them manually. The library lacks a high-level orchestration layer (e.g., `PolymarketSession` taking a wallet and returning market/order/position APIs)."

Why this matters: the gap they didn't fill is the gap my facade fills. A reviewer can compare directly. The value of this submission is precisely the orchestration layer that prior art skipped.

Source: polymarket-kit.

`@dschz/polymarket-clob-client` : *tactical fork*

Adds EIP-1193 helpers, polymorphic Node/browser crypto, smaller bundle. Tactical wins, not architectural. Shows there's appetite for ergonomic improvements over the official SDK, but I'm not competing on bundle size; I'm competing on workflow shape.

Source: @dschz/polymarket-clob-client.

Refactoring.guru / Wikipedia: *Facade vs Adapter*

Pattern	What it does	Where I use it
Facade	Simpler new interface across a *subsystem*.	`PolymarketClient` over the three Polymarket services.
Adapter	Conform to an existing interface.	`ClobAdapter` wrapping `clob-client-v2` to normalize types and error mode.

Both patterns coexist at different layers. Source: refactoring.guru.

5. Decision log

5.1 Package shape: resource-oriented facade + uniform sub-client exports

What I picked: Single facade with Stripe-style resource namespacing on top of uniform adapter sub-clients.

```
// Public surface: the facade, resource-oriented
client.markets.get({ slug })
client.orders.quote({ market, outcome, side, amountUsd })
client.orders.place({ ..., confirm: true })
client.positions.list({ market })
client.runWorkflow(input)            // top-level orchestration shortcut

// Escape hatches: uniform sub-clients (named exports)
import { GammaClient, ClobAdapter, DataApiClient } from "@poly-unified/sdk";
```

Why this shape:

- Resource namespacing maps 1:1 to Polymarket's REST organization (`/markets` , `/orders` , `/positions`), so docs-to-SDK mapping is 1:1, the way Stripe does it.
- Scales gracefully if I add `orders.cancel` , `orders.list` , `positions.history` later, without bloating the top-level namespace.
- Uniform adapter sub-clients give a coherent escape hatch: same constructor shape, same error semantics across all three.
- `runWorkflow` lives at the top level (not under a resource) because it's an *orchestration helper*, not a resource verb.

Alternatives I considered and rejected:

Alternative	Why not
Re-export `clob-client-v2` directly + thin Gamma/Data wrappers (cheapest)	Leaks `throwOnError: false` default and string-typed prices into my public API. Consistency-on-the-ladder fails.
Layered `core/` + `journeys/` (two import paths)	Added structure for structure's sake. Reads as over-architected for a prototype.
Single class, sub-clients hidden	Caged. Any non-journey use forces a fork. Violates "don't pretend concepts don't exist".
Builder/fluent (`.market().buy().usd(5).fok().confirm().place()`)	Pretty, but disproportionate effort and learning curve here.
Flat method facade (`client.getMarket()` , `client.placeMarketOrder()`)	Doesn't scale past four methods without becoming a junk drawer. Doesn't mirror REST org.

5.2 Smoke test + `dryRun` mode

What I picked: `scripts/smoke.ts` exercises Gamma + CLOB pricing + Data API positions with zero money. `client.orders.place({ ..., dryRun: true })` signs via `createMarketOrder` but skips `postOrder` , returning the signed payload.

Why:

- Reviewer's first 60 seconds: clone, set funder, `npm run smoke` , see live numbers. Strongest possible integration signal.

- `dryRun` catches what fake-CLOB unit tests can't: sigType wiring, funder mismatch, EIP-712 domain, tick precision rounding.
- `dryRun` is also genuinely useful in production for callers wanting to inspect a payload before committing.
- The smoke script can call `dryRun` mode end-to-end, proving the write path's signing without any submission.

Alternative rejected: Shipping `examples/trade.ts` (a real-trade script) in the repo is a footgun. A reviewer might run it.

5.3 Real-money guardrail: discriminated union + env gate

What I picked: Two gates answering different questions.

```
type PlaceArgs =
  | { dryRun: true;  confirm?: never; market, outcome, side, amount, ... }
  | { dryRun?: false; confirm: true;  market, outcome, side, amount, ... };
```

Plus a runtime env gate (`enableLiveTrading` config or `POLYMARKET_ENABLE_LIVE_TRADING=true`) that **only blocks live submission, not dryRun**.

Gate	Question	Layer	Bypassed by `dryRun`
`confirm: true` literal type	"Does this code intend to trade?"	Compile-time, call site	Yes
`enableLiveTrading` env/config	"Is this environment authorized to trade?"	Runtime, construction	Yes

Why two gates:

- The literal `true` type rejects `confirmRealTrade: someUserChoice` , a real footgun in the existing repo's `boolean` type.
- The env gate prevents a hardcoded `confirm: true` in a script intended for staging from trading mainnet on the wrong wallet.
- The discriminated union ensures `{ dryRun: true, confirm: true }` is a compile-time error (no need to "confirm" a non-trade), and `{ dryRun: false }` (without confirm) is also a compile-time error.

Why not just one: Earlier I argued "both gates is safety theater." The comparison artifact correctly pointed out they answer *different questions*: developer intent vs operational authorization. A single literal gate doesn't survive wrong-wallet/wrong-env scenarios; a single env gate is invisible at the call site.

5.4 DI seams + dual constructors

What I picked: `ClobClientLike` and `FetchLike` as structurally-typed injection points. Two construction paths:

```
PolymarketClient.create(config)              // production: builds viem signer, derives creds
PolymarketClient.fromDependencies(deps)      // tests: takes pre-built ClobClientLike + Fetch
```

Why: The vitest suite runs zero real network calls and zero real signing while exercising every code path. Pulled directly from the existing repo's pattern. It works, no reason to reinvent.

5.5 Tolerant Gamma normalization

What I picked: Field-level fallback resolution (`stringField` , `arrayField` , `tickSizeField` , `parseMaybeJsonArray`), snake_case/camelCase fallback, market-vs-event field hoisting. `extractMarketsPayload` flattens three response shapes; `filterMarkets` disambiguates by `conditionId` / `marketId` . Raw payload preserved on the normalized result.

Why: Polymarket's Gamma genuinely returns JSON-stringified arrays and inconsistent casing. I'd rather surface `MarketAmbiguityError` than silently pick one match.

5.6 Named error taxonomy

```
PolymarketWorkflowError                      (base)
├── PolymarketApiError(status, body)          (any HTTP error from any service)
├── MarketAmbiguityError                      (multiple markets matched a slug/event lookup)
├── MissingOutcomeError                       (caller's outcome name doesn't exist on the mar
├── LiveTradingNotEnabledError                (env gate failed)
└── OrderPlacementError                       (clob.postOrder returned failure status)
```

Why: `instanceof` discrimination beats string-matching messages. Each error class carries the structured fields a caller would want to programmatically respond to (status code, response body, ambiguity candidates, etc.).

5.7 Tick rounding direction

BUY ceils, SELL floors. The rounded "worst price" must never improve on the slippage band. Clamp to `[tickSize, 1]` .

Why: Subtle correctness point. Naive implementations round to nearest, which silently shrinks the slippage cap and can let an order go through at a worse price than the caller specified. This is the kind of bug fake-CLOB unit tests can miss; `dryRun` against a live signed payload catches it.

5.8 Workflow stitch + kill short-circuit

`client.runWorkflow(input)` runs all four steps. `validateFokOrderResponse` short-circuits before `getPositions` if the order failed/killed/rejected, preventing "positions look fine, must have worked" false-success readings.

In `dryRun` mode, `runWorkflow` still calls `getPositions` (positions are unchanged but valid to read) and returns `{ market, quote, signedPayload, positions }`. In live mode it returns `{ market, quote, receipt, positions }` only on success.

6. What I deliberately didn't do

Non-goal	Why
WebSockets / real-time data	Not needed for the journey. WS is a different ergonomic problem entirely.
GTC / GTD / FAK / batch orders / cancellations	Out of journey scope. Caller drops down to `ClobAdapter` if needed.
Deposits / withdrawals (Bridge)	Pre-trade onboarding, separate flow. This prototype assumes onboarding is done.
Neg-risk redemptions / merges / splits	Post-resolution operations. Out of journey.
Browser support / CORS workarounds	Node-only by design.
sigType 0 / 1 / 3	Hardcoded sigType 2 as the path-tested default. Configurable via override.
Multi-chain	Polygon-only by design.
Retry/backoff framework	Single-call retry on rate-limit only. Documented, not abstracted.
Balance preflight	The `not enough balance / allowance` error from CLOB is sufficient signal. Adding a preflight introduces TOCTOU.
Result-type / no-throw API	Un-idiomatic for TS. Named errors give the same discrimination with less noise.
Plugin/middleware system	Premature for a prototype.

Non-goal	Why
Schema-first codegen (à la `polymarket-kit` 's TypeBox approach)	High investment for the scope here. I hand-normalize at the edge instead. Note as a future direction.

7. Open questions

- **API cred caching across runs:** Should I persist `apiKey/secret/passphrase` to disk by default? Currently in-memory only. Disk caching needs encryption-at-rest concerns I'm not solving. The caller can pass pre-derived creds via env var.
- **Position settlement timing:** After a successful FOK, how long until `getPositions` reflects the trade? Docs say "settlement is atomic" but the trade lifecycle has `MATCHED → MINED → CONFIRMED` . May need a brief poll or doc note in the example.
- **Tick size source of truth:** Gamma returns `orderPriceMinTickSize` ; CLOB has `getTickSize(tokenId)` . I prefer Gamma (one fewer roundtrip), fall back to CLOB if missing. Worth verifying they always agree.
- **`runWorkflow` arg shape:** Should it accept the same arg shape as `orders.place` , or its own combined shape? Currently planning a stitch-specific input that combines slug + outcome + side + amount.
- **Fee transparency:** The CLOB fee formula is `C × feeRate × p × (1-p)` . Should `quoteMarketOrder` surface expected fees explicitly? Stripe always shows fees up front. Tentative yes; adds a `feeUsd` field to the quote.

8. Where each idea came from (attribution)

Idea	Source
Resource-oriented namespace (`client.orders.place`)	Stripe
`dryRun` returning a signed payload	Inspectability principle (Stripe) + extension of standard build/test patterns
Literal `confirm: true` type	AWS CDK's `requireApproval` , `rm --no-preserve-root`

Idea	Source
`enableLiveTrading` env gate (kept after the comparison)	Existing repo (`PolymarketWorkflowClient`)
Discriminated union over `dryRun` / `confirm`	Refinement during this design pass; neither prior art used this
`ClobClientLike` / `FetchLike` DI seams	Existing repo
Tolerant Gamma normalization (3 shapes, JSON-stringified arrays, snake/camel fallback)	Existing repo
Named error taxonomy	Existing repo
`validateFokOrderResponse` short-circuit	Existing repo
BUY-ceil / SELL-floor tick rounding	Existing repo
Outcome-by-name (not raw `tokenId`)	Hyperliquid
Uniform sub-client shape	CCXT
`viem.Account` as input (no custom Signer)	viem terminology
Facade + adapter pattern at different layers	Refactoring.guru
The fact that orchestration is the gap to fill	`polymarket-kit` (prior art lacks it)

9. Sources

Polymarket docs (~25 pages read)

- Docs index (llms.txt)
- Trading Quickstart · Order creation · Order lifecycle
- Authentication · Public methods · L1 methods · L2 methods
- Negative Risk markets · pUSD · Fees · Gasless
- Markets & events · Positions & tokens · Prices & orderbook
- Get market by slug · Get CLOB market info · Get order book · Get current positions
- Rate limits · Error codes · Contracts

SDKs surveyed

- @polymarket/clob-client-v2: official, the underlying CLOB wrapper I adapt

- HuakunShen/polymarket-kit: prior art with separate sub-clients, lacks orchestration (the gap I fill)
- @dschz/polymarket-clob-client: community fork with EIP-1193 helpers

External design references

- Stripe SDKs · Stripe DX teardown: Auchenberg · Stripe DX: Moesif
- CCXT docs · CCXT GitHub
- viem · viem account abstraction
- Hyperliquid TS SDK (nktkas) · QuickNode Hyperliquid SDK
- Facade pattern: Refactoring.Guru · Facade: Wikipedia · Facade vs Adapter: DevLeader
- 10 SDKs with great DX: Nordic APIs